

Lab 5 : Systolic Array with im2col for Convolution

Contents

- Goal of this lab
- Introduction
- Integrate Systolic Array to CFU Playground and Run Matmul - 20%
- im2col for Convolution - 20%
- Get Everything Together - 50%
- Questions in the Demo - 10%
- Submission

Goal of this lab

- [Integrate Systolic Array to CFU Playground and Run Matmul - 20%](#)
- [im2col for Convolution - 20%](#)
- [Get Everything Together - 50%](#)
- [Questions in the Demo - 10%](#)

Introduction

After finishing lab 3, the implementation of systolic array, we are going to integrate it into the CFU playground. As lab 3 focuses on the design of the circuit, in this lab, you will attempt to do SW-HW co-design to leverage your circuit for accelerating real-world applications.

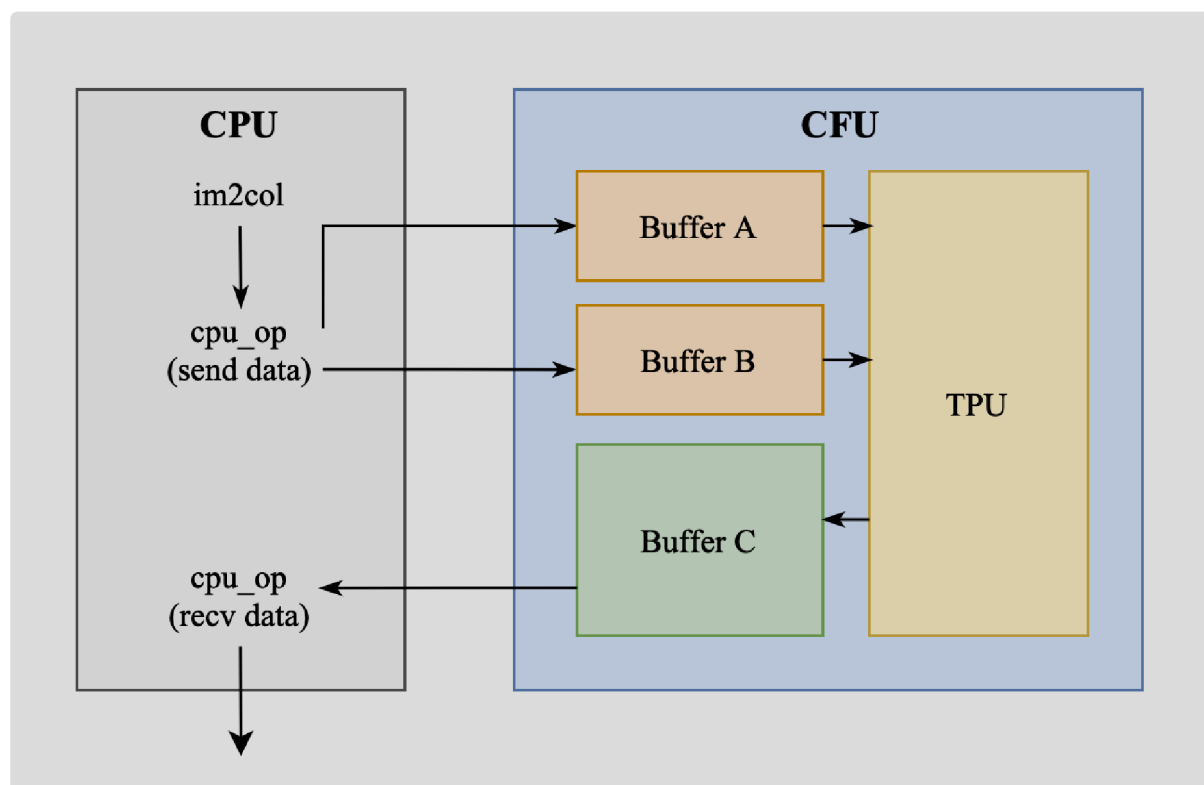
To successfully integrate your systolic array into CFU-Playground, several modifications are required to make your design synthesizable and meet the necessary constraints.

Additionally, you will develop a set of CFU operations that enable efficient communication between the CPU and the TPU for controlling your hardware.

Since the systolic array is designed for matrix multiplication, a technique known as im2col is essential to adapt it for accelerating 2D convolution operations by converting convolutions into matrix multiplications. You'll also need to implement it in this lab.

Integrate Systolic Array to CFU Playground and Run Matmul - 20%

The matrix data needs to be transmitted from the CPU to the global buffers A and B in the CFU. Once all the required data has been gathered, then the TPU will start to compute this matrix data. The outcome of the computation will be preserved in the buffer C. Finally, the data stored in the buffer C will be written back to the CPU.



The key word spotting model we intend to accelerate uses int8, and since negative values occur during computation, we need to modify it to support operations with signed integers.

You may use the below command under the lab 3 directory `AAML2025-Lab3/` to make sure it can deal with **signed** int8 correctly.

```
$ make verif_signed
```

The pattern generator will generate 50 sets of two 16*16 matrix using $\text{int8} \in [-128, 128)$ and pass it to your systolic array and check the answers.

Intergration and Verification - 20%

Now you are ready to do the intergration! Create a new project folder just like previous labs.

As shown in the block diagram, we will also be intergrating global buffer A, B and C into the CFU, we have provide a global buffer that utilises the BRAM resources of the FPGA instead of flip-flop, **please mind that this buffer does not have an async reset**.

! Important

[global_buffer_bram.v](#) for the global buffer using BRAM and the example to instantiates it.

It's totally fine if you want to use something else instead of using this BRAM buffer and example, as long as you can integrate your systolic array to into the CFU and pass the tests.

After all the hardware coding and designing the `cfu_op` to pass and recieve data from your accelerator, it is nice to do some verification now, we have provide a test before you really dive into the complicated inferencing of the KWS model.

1. Run this command under the project directory to wget the golden test

`functional_cfu_tests.cc` to your project

```
$ wget -P src/ https://github.com/nycu-caslab/AAML2025/raw/main/lab5_util/functiona
```

2. `make clean`, `make prog EXTRA_LITEX_ARGS="--cpu-variant=perf+cfu"`, `make load` and enter `2: Functional CFU Tests` from the main menu, it shall look something like this:

```
Tests for Functional CFUs
=====
h: Matmul 16*16 int8 w/ pattern 1
e: Matmul 16*16 int8 w/ pattern 2
l: Matmul 16*16 int8 w/ pattern 3
p: Matmul 16*16 int8 w/ pattern 4
!: Matmul 16*16 int8 4096 times w/ 4 patterns rotating
x: eXit to previous menu
```

`make clean` then `make prog EXTRA_LITEX_ARGS="--cpu-variant=perf+cfu"` if you don't see it.

3. Write the code about passing and receiving data to your accelerator from the software side in the `do_matmul_num` function of the `functional_cfu_tests.cc`.

! Important

- All the pattern we use to test are 16×16 int8, that is, the M, N, K are all 16.
- The input layout are just like lab3.
- For the output, you are required to place it to a 2D 16×16 int32 matrix.
- We are using the `perf_counter` here, so make sure you `make prog EXTRA_LITEX_ARGS="--cpu-variant=perf+cfu"`.
- Or you might get stuck running patterns because you don't have the `perf_counters` enabled.

For example, the first test case in the test (matrix A and its memory layout are just like lab3):

```
A:
-122  21   0 -110  44  123  -94  88  123 -110  107  22  -93  -39 -100  107
-119  -8   2  17   -1  57 -119  15 -39 -29  118 -43 -16  90  -21 -79
-118  8 -122  6  119 -93  16  96 -49 -12  92 -78  62  58  -51 -46
 70  63  41  18 -77 -57 -118  92 -98  97  94 -14  0 -120 -66 -70
 78  112 101  42 -98 -89 -99  92  1  93  91  59 -90 -93 -122  34
104  10  101 -48 -99  84  103 -88 -55  33 -81 -82 -84 -92 -83 -33
-76  15 -95 -56 -60 -66  99 -43  115 -68 -116 -59  111 -80 -69  51
 55  34  74 -105 -41  103 -70 -113 -96  0 -82 -126  45  48  126 -72
-61  126  69 -47  90  106 -10  0 -123  35  58  14 -51 -128 -120 -47
-123 -27  76 -37 -8  36  92  24 -63 -7 -20  91 -118 -60  17 -31
-18  23  19 -114 -57 -101 -27 -29  21 -61  95 -57  18 -124  19  52
 63  59  91  1  24 -87  64 -71  119 -18 -5  45 -116 -117  1  41
-28 -23 -60 -7  43  30 -95 -126 -11  119  62  89 -109 -113  41 -92
122  97 -112 -95  94 -34  50 -106 -80 -24 -33  73  126 -112  126  73
-74 -50 -37 -120 -82 -67  56  37  100  83  89 -60  69  119  45  31
107  46  4  86  6 -102  6 -67  19 -48  126  45 -38 -2 -66  80

const uint32_t A_arr[4][64] = {
{
0x86898a46, 0x15f8083f, 0x00028629, 0x92110612,
0x2cff77b3, 0x7b39a3c7, 0xa289108a, 0x580f605c,
0x7bd9cf9e, 0x92e3f461, 0x6b765c5e, 0x16d5b2f2,
0xa3f03e00, 0xd95a3a88, 0x9cebcbde, 0x6bb1d2ba,
0x4e68b437, 0x700a0f22, 0x6565a14a, 0x2ad0c897,
0x9e9dc4d7, 0xa754be67, 0x9d6763ba, 0x5ca8d58f,
0x01c973a0, 0x5d21bc00, 0x5baf8cae, 0x3baec582,
0xa6ac6f2d, 0xa3a4b030, 0x86adbb7e, 0x22df33b8,
0xc385ee3f, 0x7ee5173b, 0x454c135b, 0xd1db8e01,
0x5af8c718, 0x6a249ba9, 0xf65ce540, 0x0018e3b9,
0x85c11577, 0x23f9c3ee, 0x3aec5ffb, 0x0e5bc72d,
0xcd8a128c, 0x80c4848b, 0x88111301, 0xd1e13429,
0xe47ab66b, 0xe961ce2e, 0xc490db04, 0xf9a18856,
0x2b5eae06, 0x1edebd9a, 0xa1323806, 0x829625bd,
0xf5b06413, 0x77e853d0, 0x3edf597e, 0x5949c42d,
0x937e45da, 0x8f9077fe, 0x297e2dbe, 0xa4491f50
},

```

matrix B and its memory layout (same as lab3):

```
B:
105    -28   -103   -11    99    123   -103   -88   -59   -14   101    99    43   126   -124   62
85     74   -113    35    88    -79   118   -110  -119   -94   -68   -64  -120   -42   100   -94
-124   98    -48   -127   -74    74    51    45    52   -92    72    48   -77    87    -3   -13
-106   72    34    70     3    14   -71    85   125   41    18   -36    61   -65   123   -57
-73    21    48   113  -116  -114    96   -10   113   -56   -23    11    24  -109   -4   -23
-121  -112  -110   106  -106   106  -108  -30   126    91    68  -111   83   -88    1  -120
-29   -107  -44   -71   -45  -105  -59   -12   -36   115   -5   -27  -115  -39   108   -94
30    -74    10  -124    74    -2    41    -7   -20   -65  -33   112    38  -43   107  -14
-6   -103   85    28   -20   -93    33    21    40    92   -24    89    17  -75   -27    77
-116   80   -12   -32   115    74   -93   113   117   -95   114   -63   -84    -6   -57    75
-97    29   -77   -89    89   -79  -122   -93   -27    78   100   -90   125   -17   124   103
-11   -121   49  -103   -19     9    24   114    94  -123   -30   -67   -75    13    63   -44
101    22    28   115    14    89   -12     1   109   -71   -94   101   -42   -32   -90   -46
-65     7   -86   -77   -81   -94    60   -22    92    54    88   -55  -104   -92   -33    21
-53  -122   120    10    73  -127   -97    99    28   -25   122   117   109    71    60   -37
78    -26   -26   110  -123   -92    -7   115   -30   -21   -62   -70   -99   -90    14  -111
```

```
const uint32_t B_arr[4][64] = {
{
0x69e499f5, 0x554a8f23, 0x8462d081, 0x96482246,
0xb7153071, 0x8790926a, 0xe395d4b9, 0x1eb60a84,
0xfa99551c, 0x8c50f4e0, 0x9f1db3a7, 0xf5873199,
0x65161c73, 0xbf07aab3, 0xcb86780a, 0x4ee6e66e,
0x637b99a8, 0x58b17692, 0xb64a332d, 0x030eb955,
0x8c8e60f6, 0x966a94e2, 0xd397c5f4, 0x4afe29f9,
0xeca32115, 0x734aa371, 0x59b186a3, 0xed091872,
0x0e59f401, 0xafa23cea, 0x49819f63, 0x85a4f973,
0xc5f26563, 0x89a2bcc0, 0x34a44830, 0x7d2912dc,
0x71c8e90b, 0x7e5b4491, 0xdc73fbe5, 0xecbdf70,
0x285ce859, 0x75a172c1, 0xe54e64a6, 0x5e85e2bd,
0x6db9a265, 0x5c3658c9, 0x1ce77a75, 0xe2ebc2ba,
0x2b7e843e, 0x88d664a2, 0xb357fdf3, 0x3dbf7bc7,
0x1893fce9, 0x53a80188, 0x8dd96ca2, 0x26d56bf2,
0x11b5e54d, 0xacfac74b, 0x7def7c67, 0xb50d3fd4,
0xd6e0a6d2, 0x98a4df15, 0x6d473cdb, 0x9da60e91
},

```

the answer should be :

```
C:
-3905  -26185  -9367   6402  -42569  -30078   32438  -18662  -15463  19579  -29788  -29902  20468  -45106  25667  -4880
-38038  24594  -10103  -9424  -2609   -3175   5756  -20622  17392  15354  15454  -26106  32280  -18115  15588  17578
2991   21852  10405   3993   7881  -42912  25318  -26126  -1485   7216  -26193  -2648   8555  -41379  25270  12345
10623  49907  -20197  -32177  71845  56703  -13832  -19842  -27933  -42898  10469   4611  19080  40268   3973  32993
10091  44927  -28941  -49505  53133  33986   5962   -4294  -36784  -44364   3143  -12874  -14216  27901  19844  28309
352    12644  -44120  -9642   -1489  56322  -22434  -21250  -33439  16434  17772  -11833  -18392  40224  -26367  -7151
62361  -15919  25397  39675  -15622  -14436  25552  -2606  -31534  21334  -68124  21029  -32426  -17188  -17520  -16590
-1164  14705  -21873  21511   3084  35640  -11553  -20795  -1904   -6086  36421  16916   9051  38285  -50537  -6044
-21614  34799  -38018   1597  -3368  25647  16636  -30288  -10156  -26952  -11293  -47776  -5619  -5339  23198  -19867
-35640  -19536  8092  -39753  -20332  -9299   6466  21237  -1584  -4025   1391  -23271  -10720  7320  38884  -21547
37377   9477   8244  -3540  22271  -13960   3016  -16832  -57246  -2944  -18289  18029  10995  27166     0  18776
5427   -3600   8495  -12458   135  -23524  11335   7577  -32606  -4280  -4094   8160  -15395  21619  12033   5595
-38251   5613  21011   4696  26074  12250  -26729  15843  16779  -10510  28342  -31908  36207  19758  -1873  29594
62319  -24530   9599  52589  16794  -13681  -10282  -3841  -23269  -36445  -22434  16489  -9990  19385  -18156  -31465
-67   -14606  6311  -28534  14889  -37619  -9055   3879  -2605  22138  12247  10785  -17301  -14626  -7383  31528
14020  20392  -19111  -6164   8922  -18005  -6027  -12530  -24481   5924   328  -18223  -4119   4097  16579  13961
```

```

void do_matmul_num(int test_num) {
    // place your answer in this array!
    uint32_t C_arr[16][16];

    // =====
    // Implement your design here,
    // and DO NOT MODIFY ANYTHING "ABOVE" THIS LINE !!
    // =====

    /**
     * All inputs are 16*16 signed int8 matrix, outputs are 16*16 signed int32.
     * 1. Pass data to CFU.
     * 2. Pass matrix parameters to CFU.
     * 3. Receive data from CFU and place it to `C_arr`.
     */

    // =====
    // DO NOT MODIFY ANYTHING "BELOW" THIS LINE !!
    // =====

    for (uint32_t i = 0; i < 16; i++) {
        for (uint32_t j = 0; j < 16; j++) {
            if (C_arr[i][j] != C_arr_ans[test_num][(i<<4)+j]) {
                error_ct++;
                printf("*** %ld error(s) @ pattern no. %d\n ---> golden C_arr[%02ld][%02ld]\n",
                    error_ct, test_num, i, j, C_arr_ans[test_num][(i<<4)+j], i, j, C_arr[i][j]);
            }
        }
    }
}

```

Evaluation Criteria

If you successfully complete the final part [Get Everything Together](#), you will automatically receive the score for this section. However, if you are unable to complete the final part, you can test this section independently and still earn the score for it.

! Important

You will get **0%** if you can't pass the golden test or didn't meet the requirement, you have to pass the pattern below to get 20%.

Passes pattern	4096 times 1-4 rotating
cycle ct. requirement	< 45M

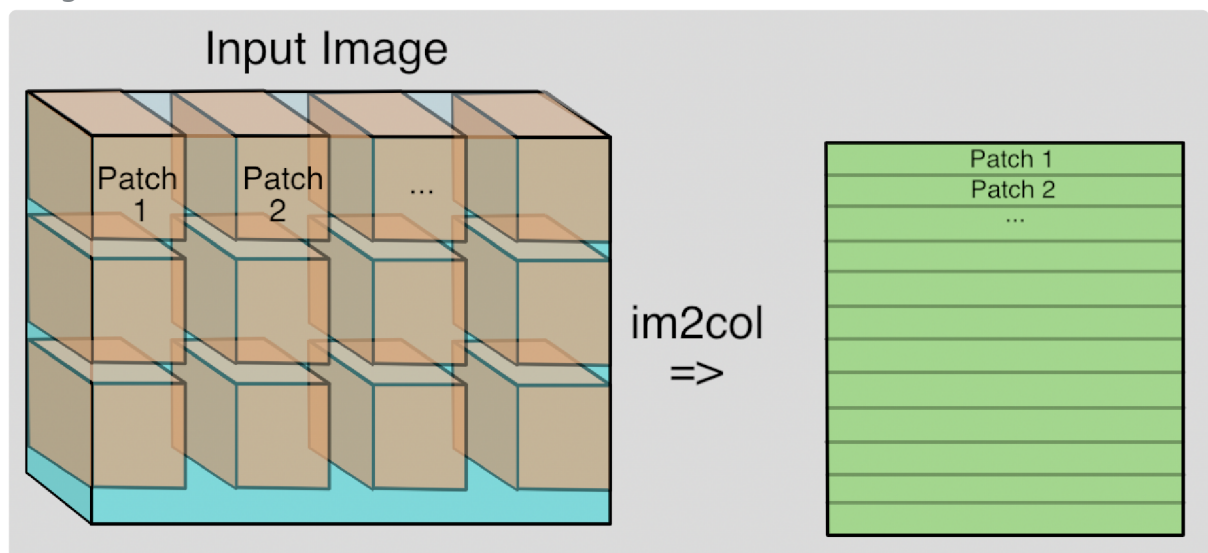
im2col for Convolution - 20%

Using the im2col technique, we can convert a 2D convolution operation into the multiplication of two matrices, which can then be efficiently accelerated by the systolic array.

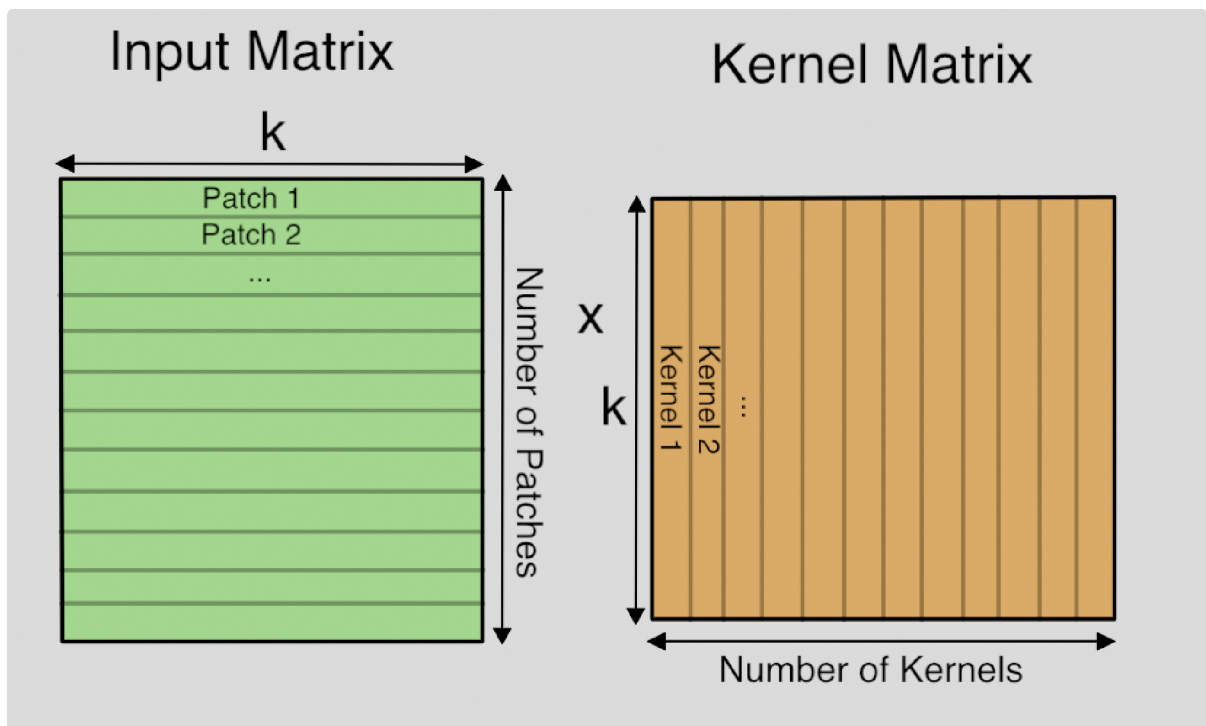
To construct these two matrices, which we will refer to as the kernel matrix and the im2col matrix, you can refer to the article: [Why GEMM is at the heart of deep learning](#)

Here is the core part I excerpted from the article:

The first step is to turn the input from an image, which is effectively a 3D array, into a 2D array that we can treat like a matrix. Where each kernel is applied is a little three-dimensional cube within the image, and so we take each one of those cubes of input values and copy them out as a single column into a matrix. This is known as im2col, for image-to-column.



Now you have the input image in matrix form, you do the same for each kernel's weights, serializing the 3D cubes into rows as the second matrix for the multiplication. Here's what the final GEMM looks like:



Here 'k' is the number of values in each patch and kernel, so it's kernel width * kernel height * depth. The resulting matrix is 'Number of patches' columns high, by 'Number of kernel' rows wide. This matrix is actually treated as a 3D array by subsequent operations, by taking the number of kernels dimension as the depth, and then splitting the patches back into rows and columns based on their original position in the input image.

i Note

Other than the shapes of convolution, it is also crucial to handle parameters such as **stride**, **dilation** and **padding** correctly to ensure the matrices are constructed properly for the convolution.

For simplicity, in this lab, **you can ignore batch and group** parameters, assuming both are set to 1. This assumption will still allow you to pass all the required tests.

Also, you may have noticed that dynamic memory allocation, such as using new or malloc, is not supported in CFU-Playground. Therefore, we recommend declaring fixed-size arrays, for example:

```
int8_t im2col[...][...];  
int8_t kernel[...][...];  
int32_t result[...][...];
```

Once the kernel matrix and im2col matrix are constructed, you can perform a simple matrix multiplication (naive GEMM) to verify your implementation in this part.

Afterward, you will need to reconstruct the output tensor from the result matrix. This process shares similar concepts with constructing the im2col and kernel matrices. Finally, don't forget to apply post-processing to the tensor, such as quantization and bias addition.

Under your project directory, add the `conv.h` to your project and do your modification on the `ConvPerChannel` function.

```
$ cp \
  ../../third_party/tflite-micro/tensorflow/lite/kernels/internal/reference/integer
src/tensorflow/lite/kernels/internal/reference/integer_ops/conv.h
```

Download the unit tests for testing your implementation under your lab5 project directory:

```
$ cd ${CFU_ROOT}/proj/lab5_proj
$ mkdir -p src/tensorflow/lite/micro/kernels
$ wget -P src/tensorflow/lite/micro/kernels \
  https://github.com/nycu-caslab/AAML2025/raw/main/lab5_util/conv_test.cc
$ wget -P src/ \
  https://github.com/nycu-caslab/AAML2025/raw/main/lab5_util/tflite_unit_tests.cc
```

Evaluation Criteria

If you successfully complete the final part [Get Everything Together](#), you will automatically receive the score for this section. However, if you are unable to complete the final part, you can test this section independently and still earn the score for it.

To earn the score for this part, you need to perform the im2col transformation for the convolution. However, the matrix multiplication can be handled purely through software using a naive GEMM implementation. To pass, you must successfully complete the `TFLite Unit Tests` without any failures.

```
CFU Playground
=====
1: TFLM Models menu
2: Functional CFU Tests
3: Project menu
4: Performance Counter Tests
>5: TFLite Unit Tests
6: Benchmarks
7: Util Tests
8: Embench IoT
main> 5
```

12/12 tests passed
~~~ALL TESTS PASSED~~~

All 12 tests must be passed.

## Get Everything Together - 50%

Now you have a working systolic array that can do matmul correctly, and a im2col SW code for turning convolution into matmul, it's time to get everything together!

Since the the buffer size of systolic array is limited, you might need **tiling** for the matrix multiplication. If you are not familiar with it, you can refer to this article:

[How to tile matrix multiplication](#)

Here's a simple example for you. Here, T represents the size of each tile, which you may want to set based on the buffer size. In addition to the outer loops, some inner loops may be required to manage the sending and receiving logic.

```
constexpr int T; // tile size

for (int m = 0; m < pad_M; m += T) {
    for (int n = 0; n < pad_N; n += T) {
        for (int k = 0; k < pad_K; k += T) {
            // send to im2col buffer
            // send to kernel buffer
            // receive from output buffer
        }
    }
}
```

You may run into some boundary problems when padding. **Handle the values that are out of boundary correctly to achieve the correct result.**

Please note that there are `input_offset` when doing convolution, you may either modify the hardware to accumulate with the offset, or try any method as you see fit.

```
acc += filter_val * (input_val + `input_offset`)
```

### ! Important

Add the sixth perf counter at the beginning and end of the topmost

`ConvPerChannel` function within the `conv.h` file in your project. This step is crucial for evaluating your score, so please ensure it is not overlooked.

```
// Fixed-point per-channel-quantization convolution reference kernel.
inline void ConvPerChannel(
    const ConvParams& params, const int32_t* output_multiplier,
    const int32_t* output_shift, const RuntimeShape& input_shape,
    const int8_t* input_data, const RuntimeShape& filter_shape,
    const int8_t* filter_data, const RuntimeShape& bias_shape,
    const int32_t* bias_data, const RuntimeShape& output_shape,
    int8_t* output_data) {
    perf_enable_counter(6);

    ...

    perf_disable_counter(6);
}
```

## Evaluation Criteria

We will test the performance of your implementation based on the quantized KWS model provided in lab2. Note that your systolic array + im2col shall be faster than lab2's SIMD method, we will be running several labels to make sure you have done everything right.

Also, to earn the score for this part, you still need to complete the `TFLite Unit Tests` without any failures.

### ! Important

You will get **0%** if you can't pass the `TFLite Unit Tests` or didn't meet the requirement.

| Cycles of Counter 6 | > 650M | 650~550M | 550~480M | < 480M |
|---------------------|--------|----------|----------|--------|
| Score               | 0      | 30       | 40       | 50     |

## Questions in the Demo - 10%

You will be asked several questions about the concepts covered in this lab and your implementation. This section accounts for 10% of the total lab score.

## Submission

---

You need to hand in your **CFU-Playground project folder** without the `build` folder and renamed with your student ID.

Please organize your submission files into a zip archive structured as follows:

```
YourID.zip
├── YourID/
│   ├── src/
│   │   ├── folder...
│   │   └── files...
│   ├── cfu.v
│   └── Makefile
```

### ! Important

TAs should be able to run your project without any modification. If TAs cannot compile or run your code, **you won't receive any points, even if you passed the DEMO**. Also, **PLAGIARISM is not allowed**.